

Compendio Esami Machine Learning

Tutte le Domande, Esercizi e Linee Guida di Risoluzione

Basato sulle slide e sugli esami del Prof. Luca Iocchi

Indice

1	Supervised Learning: Linear Models (Classification & Regression)	2
1.1	Support Vector Machines (SVM) e Massimizzazione del Margine	2
1.2	Perceptron Model	2
1.3	Regression, Linear Models e Kernel Trick	3
2	Supervised Learning: Decision Trees (DT)	4
2.1	ID3, Entropia e Information Gain	4
2.2	Overfitting e Pruning	4
3	Supervised Learning: Probabilistic & Bayesian Learning	5
3.1	MAP, Maximum Likelihood (ML) e Bayes Optimal Classifier	5
3.2	Naive Bayes Classifier	6
3.3	Generative vs Discriminative Probabilistic Models	6
4	Supervised Learning: Artificial Neural Networks (ANN, CNN, Autoencoders)	7
4.1	Backpropagation e Ottimizzazione SGD	7
4.2	Loss Functions e Output Activations	7
4.3	Convolutional Neural Networks (CNN) e Dropout	8
4.4	Autoencoders	8
5	Ensemble Methods & Instance-Based Learning	9
5.1	Ensemble: Bagging, Boosting e AdaBoost	9
5.2	K-Nearest Neighbors (K-NN)	9
6	Performance Evaluation	10
6.1	K-Fold Cross Validation	10
6.2	Confusion Matrix, Accuracy, Precision, Recall	10
7	Unsupervised Learning: Clustering & Dimensionality Reduction	11
7.1	K-Means Algorithm	11
7.2	Gaussian Mixture Models (GMM) e EM	12
7.3	Principal Component Analysis (PCA)	12
8	Reinforcement Learning (RL) & MDPs	13
8.1	Markov Decision Process (MDP) Model	13
8.2	Q-Learning e Apprendimento	13
8.3	Exploitation vs Exploration e k-Armed Bandit	14

1 Supervised Learning: Linear Models (Classification & Regression)

1.1 Support Vector Machines (SVM) e Massimizzazione del Margine

Domande d'esame raccolte:

- Describe the principle of maximum margin used by SVM classifiers through its formal mathematical definition.
- Draw a linearly separable dataset for 2D binary classification. Draw a possible solution obtained by SVM and highlight the margin and the support vectors.
- Discuss why the maximum margin solution is preferred for the classification problem.
- Describe the principle of soft margins used by SVM classifiers. Illustrate the concept with a geometric example.
- Consider a classification problem based on SVMs. Explain if the data in the figure are linearly separable and what type of kernel function you would choose to obtain perfect separability.

Linea Guida di Risoluzione (Rif. Slide 7 - Linear models for classification):

- **Definizione Matematica del Margine:** L'obiettivo della SVM è trovare l'iperpiano separatore $y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 = 0$ che massimizza il *margin*, ovvero la distanza minima tra l'iperpiano e i punti del dataset più vicini ad esso. La distanza di un punto $\mathbf{x}_n$ dall'iperpiano è data da $\frac{|y(\mathbf{x}_n)|}{\|\mathbf{w}\|}$.
- **Problema di Ottimizzazione (Hard Margin):** Massimizzare il margine equivale a minimizzare la norma dei pesi. Si risolve il problema:

$$\arg \min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2$$

soggetto ai vincoli per ogni campione: $t_n(\mathbf{w}^T \mathbf{x}_n + w_0) \geq 1 \quad \forall n$.

- **Soft Margins (Dati non linearmente separabili):** Se i dati presentano rumore o sovrapposizioni, si introducono le *slack variables* $\xi_n \geq 0$. Il problema diventa:

$$\arg \min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n$$

soggetto a $t_n(\mathbf{w}^T \mathbf{x}_n + w_0) \geq 1 - \xi_n$. Il parametro C regola il trade-off tra la penalità per l'errore e l'ampiezza del margine.

- **Perché preferire SVM:** Diminuisce la probabilità di misclassificare campioni futuri (migliore generalizzazione). Metodi come il Perceptrone si fermano su una qualsiasi retta separatrice, che spesso risulta troppo vicina ad alcuni campioni.
- **Support Vectors:** Sono gli unici punti del dataset per i quali vale l'uguaglianza $t_n(\mathbf{w}^T \mathbf{x}_n + w_0) = 1$ (o $1 - \xi_n$). Solo essi contribuiscono alla definizione dell'iperpiano.

1.2 Perceptron Model

Domande d'esame raccolte:

- Describe the perceptron model for classification and its training rule.
- Draw a graphical representation of a linearly separable 2D data set and provide a qualitative graphical example of a possible evolution of perceptron training (4 images showing the temporal evolution).

- Discuss the difference between two solutions (Perceptron vs SVM) and explain why the SVM is often preferred.

Linea Guida di Risoluzione (Rif. Slide 7):

- **Modello:** È un classificatore binario la cui uscita è determinata dalla funzione *sign*: $o(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0)$.
- **Training Rule (Discesa del Gradiente):** L'obiettivo è minimizzare l'errore quadratico dei campioni misclassificati. La regola di aggiornamento dei pesi è:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E \implies \mathbf{w} \leftarrow \mathbf{w} + \eta \sum_{n=1}^N (t_n - o_n) \mathbf{x}_n$$

dove η è il *learning rate* (step size).

- **Evoluzione grafica:** 1) Disegnare i punti base. 2) Inserire una retta casuale (step 1). 3) Evidenziare un punto misclassificato: il vettore normale alla retta $\mathbf{w}$ si "ruota" verso il punto (se la classe corretta è +1) o si "allontana" (se -1). 4) Un η troppo piccolo porta a una convergenza lentissima, un η grande fa oscillare bruscamente la retta separatrice attorno alla soluzione.

1.3 Regression, Linear Models e Kernel Trick

Domande d'esame raccolte:

- Define a regularized squared error function to solve the linear regression problem.
- Give a short explanation of the Kernel trick (kernel substitution). What are the necessary conditions for applying it?
- Provide the formal definition of the Gram matrix.
- Describe an iterative approach to solve the problem (Gradient Descent).
- Logistic regression is a probabilistic discriminative model. What function does it optimize?

Linea Guida di Risoluzione (Rif. Slide 8 e 9):

- **Regularized Error Function per Regression:**

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (t_n - y(\mathbf{x}_n))^2 + \lambda \|\mathbf{w}\|^2$$

dove $y(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x})$. Il termine $\lambda \|\mathbf{w}\|^2$ (L2 regularization) serve ad attenuare l'*overfitting*. L'aggiornamento iterativo usa SGD: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E$.

- **Kernel Trick:** Permette di trovare separatori lineari in spazi ad altissima dimensionalità senza calcolare esplicitamente le feature in quello spazio (la funzione $\phi(\mathbf{x})$). **Condizione necessaria:** L'algoritmo (sia in training che in prediction) deve poter essere riscritto in modo che le osservazioni in input $\mathbf{x}$ compaiano *esclusivamente* all'interno di prodotti scalari $\mathbf{x}^T \mathbf{x}'$.
- Se la condizione è rispettata, si sostituisce il prodotto scalare con una funzione kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$.
- **Gram Matrix:** È la matrice K quadrata di dimensione $N \times N$ (dove N è il numero totale di campioni nel training set). Gli elementi sono dati dalle valutazioni del kernel: $K_{i,j} = K(\mathbf{x}_i, \mathbf{x}_j)$.
- **Logistic Regression:** Ottimizza (minimizza) la **Cross-Entropy function** (negative log-likelihood). A differenza del Perceptrone, genera una stima probabilistica. Spesso addestrata tramite *Iterative Reweighted Least Squares* (IRLS), derivato dal metodo di ottimizzazione di Newton-Raphson.

2 Supervised Learning: Decision Trees (DT)

2.1 ID3, Entropia e Information Gain

Domande d'esame raccolte:

- Explain the ID3 algorithm for Decision Tree learning: formally describe input, output and the main steps of the algorithm.
- Simulate the execution of the ID3 algorithm on a dataset and generate the corresponding output tree (compute Information Gain).
- Provide a rule-based representation of the tree T .
- Provide a formal definition of consistency of an hypothesis with respect to a dataset. Determine if tree T is consistent.

Linea Guida di Risoluzione (Rif. Slide 3 - Decision Trees):

- **ID3 Algorithm (Input/Output/Steps):**
 - *Input:* Set di Esempi D , Attributo Target c , Set di Attributi A . *Output:* Un Albero Decisionale.
 - *Steps:* 1) Crea nodo radice. 2) Se tutti gli esempi in D hanno la stessa classe c_i , ritorna una foglia con etichetta c_i . 3) Se il set di attributi è vuoto, ritorna foglia con classe maggioritaria. 4) Altrimenti, scegli l'attributo A^* che **massimizza l'Information Gain**. 5) Per ogni valore v di A^* , crea un ramo e richiama l'algoritmo ricorsivamente sul subset di dati dove $A^* = v$.
- **Calcoli Matematici:**
 - **Entropy(S)** = $-p_+ \log_2(p_+) - p_- \log_2(p_-)$. È la misura dell'impurità (incertezza) del dataset.
 - **Information Gain(S, A)** = $\text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$. Scegliere sempre l'attributo col Gain più alto.
- **Rule-Based Representation:** Scrivere le regole sotto forma di disgiunzione (OR) di congiunzioni (AND). Es:
 $(\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{Normal}) \vee (\text{Outlook} = \text{Overcast}) \implies \text{PlayTennis} = \text{Yes}$.
- **Consistency:** Un'ipotesi h è coerente (*consistent*) con un dataset D se e solo se $h(x) = c(x) \quad \forall (x, c(x)) \in D$. (Per verificarlo nell'esercizio, passa i campioni forniti dentro l'albero: se l'etichetta foglia combacia con quella reale per tutti, è consistent).

2.2 Overfitting e Pruning

Domande d'esame raccolte:

- Provide a formal definition of overfitting in learning with Decision Trees.
- Describe the concept of pruning in Decision Tree learning, highlighting the main motivation and an operational procedure to perform it (e.g. Reduced-Error Post-Pruning).

Linea Guida di Risoluzione (Rif. Slide 3):

- **Overfitting Formal Definition:** Un'ipotesi $h \in H$ fa *overfitting* sul training data se esiste un'ipotesi alternativa $h' \in H$ tale che $\text{error}_{\text{train}}(h) < \text{error}_{\text{train}}(h')$ ma, considerando la distribuzione generale dei dati, si ha $\text{error}_{\text{true}}(h) > \text{error}_{\text{true}}(h')$.

- Negli alberi decisionali accade quando l'albero è troppo profondo (es. una foglia per ogni singolo campione), finendo per "memorizzare" il rumore.
- **Pruning (Reduced-Error Post-Pruning):** La motivazione è semplificare il modello per favorire la capacità di generalizzazione sui dati futuri.
- *Procedura:* 1) Dividere il dataset in *Training Set* e *Validation Set*. 2) Addestrare un albero completo sul Training Set (lasciandolo fare overfitting). 3) Finché l'accuratezza sul Validation Set non peggiora: iterare su ogni nodo interno, provare a rimuovere il suo sotto-albero assegnando la classe maggioritaria; selezionare il taglio (pruning) che porta al **maggior aumento di accuratezza** sul Validation Set. L'algoritmo si ferma quando nessun taglio migliora le performance.

3 Supervised Learning: Probabilistic & Bayesian Learning

3.1 MAP, Maximum Likelihood (ML) e Bayes Optimal Classifier

Domande d'esame raccolte:

- Define Maximum a posteriori (MAP) hypotheses and Maximum likelihood (ML) hypotheses.
- Comment the following statement: "In a classification problem, the class returned by the ML hypothesis on a new instance x is always the most probable class".
- Define the concept of Bayes Optimal Classifier and discuss its practical applicability.
- Given $P(D|h_1) = 0.6$, $P(D|h_2) = 0.8$ and priors $P(h_1) = 0.2$, $P(h_2) = 0.1$, determine the higher a posteriori hypothesis.

Linea Guida di Risoluzione (Rif. Slide 5 - Bayesian Learning):

- **Formule Base (Teorema di Bayes):** $P(h|D) = \frac{P(D|h)P(h)}{P(D)}$.
- **MAP Hypothesis:** Sceglie l'ipotesi più probabile dati i dati, considerando la probabilità a priori $P(h)$.

$$h_{MAP} = \arg \max_{h \in H} P(D|h)P(h).$$
- **ML Hypothesis:** Sceglie l'ipotesi che rende i dati osservati più probabili.

$$h_{ML} = \arg \max_{h \in H} P(D|h).$$
 È equivalente a MAP *solo se* assumiamo che le probabilità a priori $P(h)$ siano distribuite uniformemente (tutte uguali).
- **Risposta al commento:** L'affermazione è **Falsa**. L'ipotesi ML ignora del tutto la distribuzione a priori. Se una classe è a priori molto rara, ML potrebbe comunque sceglierla basandosi solo sulla verosimiglianza dei dati. L'ipotesi più probabile in assoluto è fornita dal calcolo MAP o, ancor meglio, dal **Bayes Optimal Classifier**.
- **Bayes Optimal Classifier:** A differenza di MAP che sceglie la *singola ipotesi* migliore, questo classifica un nuovo campione x combinando le predizioni di **tutte le ipotesi** pesate per la loro probabilità a posteriori:

$$c_{opt} = \arg \max_{c \in C} \sum_{h \in H} P(c|h, x)P(h|D)$$

Praticabilità: Offre le prestazioni ottime in assoluto, ma non è pratico in sistemi reali poiché richiede di iterare su *tutte* le ipotesi dello spazio H , che in genere sono infinite o esponenziali.

- **Esercizio Pratico:** Calcola $P(D|h_1)P(h_1) = 0.6 \times 0.2 = 0.12$. Calcola $P(D|h_2)P(h_2) = 0.8 \times 0.1 = 0.08$. Poiché $0.12 > 0.08$, l'ipotesi a posteriori più alta è h_1 .

3.2 Naive Bayes Classifier

Domande d'esame raccolte:

- Formally describe the concept of Naive Bayes Classifier. Describe the assumption made and comment on its practical applicability.
- Describe how to predict the class for a new sample based on Naive Bayes (which terms are computed and how?).
- Application to text categorization (Spam vs Non-Spam, Appropriate vs Inappropriate posts).

Linea Guida di Risoluzione (Rif. Slide 5 e 6):

- **Formulazione Naive Bayes:** Calcola la classe più probabile tramite:

$$c_{NB} = \arg \max_{c \in C} P(c) \prod_{i=1}^d P(x_i|c)$$

- **Assunzione (Indipendenza Condizionata):** Il Naive Bayes assume che gli attributi x_i siano **statisticamente indipendenti** l'uno dall'altro data la classe c .
- **Applicabilità Pratica:** Nonostante l'assunzione di indipendenza sia spessissimo violata nella realtà (es. in un testo la parola "San" e "Francisco" compaiono quasi sempre assieme), il classificatore **funziona incredibilmente bene** nella pratica per task come la classificazione del testo. Riduce radicalmente la complessità computazionale e il problema dei dati sparsi, richiedendo il calcolo di $O(d)$ termini invece di $O(2^d)$.
- **Applicazione ai testi (Bag of words):** $P(c)$ è la frequenza della classe nel dataset. I termini $P(x_i|c)$ si stimano contando il numero di documenti della classe c che contengono la parola x_i fratto il numero totale di documenti in classe c . Spesso si applica *Laplace Smoothing* (+1 al numeratore) per evitare probabilità a zero.

3.3 Generative vs Discriminative Probabilistic Models

Domande d'esame raccolte:

- Describe a probabilistic generative model for a classification problem, assuming Gaussian distributions. Identify the parameters and determine the size of the model.
- Describe the difference between generative and discriminative probabilistic models for classification.

Linea Guida di Risoluzione (Rif. Slide 6):

- **Generative Models (es. Gaussian Mixture, Naive Bayes):** Modellano la distribuzione congiunta $P(X, Y)$ calcolando prima $P(X|Y)$ e $P(Y)$. Permettono di "generare" nuovi campioni campionando dalle distribuzioni apprese.
- **Discriminative Models (es. Logistic Regression):** Modellano direttamente la probabilità a posteriori $P(Y|X)$ ottimizzando un boundary decisionale tra le classi, senza calcolare la distribuzione dei dati in ingresso.
- **Parametri Modello Gaussiano Generativo (K Classi):** Assumiamo $P(x|C_i) = \mathcal{N}(x; \mu_i, \Sigma_i)$ e prob. a priori p_i . Per K classi, ci sono: $(K - 1)$ probabilità a priori p_i ; K vettori medie μ_i (ciascuno di dimensione d); K matrici di covarianza Σ_i (ognuna $d \times d$). *Esempio per $K=3$, spazio $d=2$ (2D data):* p_i : $3 - 1 = 2$ parametri liberi. μ_i : $3 \times 2 = 6$ parametri. Σ : se diagonale, $3 \times 2 = 6$; se piene simmetriche, $3 \times \frac{2(2+1)}{2} = 9$ parametri. Totale: $2 + 6 + 9 = 17$ parametri.

4 Supervised Learning: Artificial Neural Networks (ANN, CNN, Autoencoders)

4.1 Backpropagation e Ottimizzazione SGD

Domande d'esame raccolte:

- Describe the role of Backpropagation and Stochastic Gradient Descent (SGD).
- Provide the main steps of the SGD algorithm, highlight the hyperparameters, and discuss the sensitivity of the solution with respect to them.
- Is Backpropagation affected by local minima or overfitting?

Linea Guida di Risoluzione (Rif. Slide 11 - Neural Networks):

- **Backpropagation (Role):** È un efficiente algoritmo analitico basato sulla *chain rule* delle derivate, usato per **calcolare il gradiente** della funzione di Loss rispetto a tutti i pesi della rete $\nabla_{\theta} J$. **NON è l'algoritmo di training in sé**, né è soggetto a minimi locali, poiché è solo un calcolo matematico del gradiente.
- **SGD (Role & Steps):** È l'algoritmo iterativo di ottimizzazione che sfrutta il gradiente per aggiornare i pesi e addestrare la rete. *Steps:* 1) Campiona un mini-batch random dal dataset D . 2) Fai Forward pass e computa la Loss. 3) Usa Backprop per ottenere il gradiente. 4) Aggiorna i parametri: $\theta \leftarrow \theta - \eta \nabla_{\theta} J$.
- **Iperparametri e Sensibilità:** Il parametro chiave è il **learning rate** η . SGD è affetto dai minimi locali (perché la loss delle ANN è fortemente non-convessa). Se η è troppo grande, SGD salta il minimo e diverge o oscilla bruscamente. Se η è troppo piccolo, la convergenza diventa estremamente lenta o si incastra nel primo minimo locale incontrato. Altri iperparametri includono *batch size* e *momentum* (che aiuta ad accelerare l'uscita da minimi locali). SGD è soggetto ad *overfitting* se allenato per troppe epoche senza regolarizzazione.

4.2 Loss Functions e Output Activations

Domande d'esame raccolte:

- What is a suitable activation function for the output layer of the network for Regression / Binary / Multiclass problems? Explain.
- For the selected activation function explain if the output units saturate and how learning parameters is affected.

Linea Guida di Risoluzione (Rif. Slide 11):

- **Regressione** ($f : \mathbb{R}^d \rightarrow \mathbb{R}$): Attivazione: **Lineare** (Identity). Loss: **MSE** (Mean Squared Error).
- **Binary Classification** ($f : \mathbb{R}^d \rightarrow \{0, 1\}$): Attivazione: **Sigmoide** ($\sigma(x) = \frac{1}{1+e^{-x}}$). Rende l'output una probabilità differenziabile. Loss: **Binary Cross-Entropy**.
- **Multiclass Classification:** Attivazione: **Softmax**. Assicura che la somma delle probabilità in uscita faccia 1. Loss: **Categorical Cross-Entropy**.
- **Saturazione:** Usando Sigmoide/Softmax, l'unità satura ad alte magnitudini dell'input, portando la derivata (gradiente) quasi a zero (*vanishing gradient*). Accoppiando queste funzioni con la Loss Cross-Entropy, il calcolo della derivata annulla il termine di saturazione, facendo in modo che la rete si aggiorni correttamente e l'unità saturi *solo* quando sta dando la risposta decisamente corretta.

4.3 Convolutional Neural Networks (CNN) e Dropout

Domande d'esame raccolte:

- Consider a layer of a CNN with input feature maps of size $96 \times 96 \times 10$. Design the layer (kernel, pooling) to obtain output feature maps of $30 \times 30 \times 25$.
- Compute the number of trainable parameters produced by the layer.
- Describe properties of sparse connectivity and parameter sharing.
- Explain what is the role of the dropout layer and the meaning of the dropout rate.

Linea Guida di Risoluzione (Rif. Slide 12 - CNNs):

- **Dimensioni Mappe (Formula):** La dimensione spaziale (width/height) in output dopo una convoluzione/pooling è data da:

$$W_{out} = \frac{W_{in} - W_k + 2P}{S} + 1$$

dove W_k è la dimensione del kernel, P il padding e S lo stride.

- **Risoluzione Esempio numerico:** Input 96. Vogliamo Output 30. Senza padding ($P = 0$) e $S = 3$: $30 = \frac{96 - W_k}{3} + 1 \implies 29 = \frac{96 - W_k}{3} \implies W_k = 96 - 87 = 9$. Quindi serve un kernel 9×9 con stride 3. Se serve una depth in output di 25, si applicheranno $d_{out} = 25$ kernels.
- **Numero Parametri Addestrabili:** La formula per il numero di pesi di un layer convoluzionale è:

$$|\theta| = (w_k \cdot h_k \cdot d_{in} + 1) \cdot d_{out}$$

(Il +1 è il parametro di Bias per ogni kernel). *Numerico:* Kernel 9×9 , Input Depth = 10, Output Depth = 25.

$$(9 \times 9 \times 10 + 1) \times 25 = (810 + 1) \times 25 = 811 \times 25 = 20275 \text{ parametri.}$$

- **Sparse Connectivity & Parameter Sharing:** Invece di una matrice densa (Fully Connected), i neuroni sono connessi solo a un piccolo campo recettivo (*sparse connectivity*). Lo stesso set di pesi del kernel è fatto "scorrere" su tutta l'immagine (*parameter sharing*). Questo riduce drasticamente i parametri, allevia l'overfitting e conferisce l'abilità di riconoscere pattern (*translation invariance*) ovunque si trovino nell'immagine.
- **Dropout:** Tecnica di regolarizzazione. Il *dropout rate* (es. 0.5) rappresenta la probabilità con cui durante il training, a ogni forward pass, un'unità della rete viene disattivata (ignorata). Riduce il co-adattamento tra i neuroni agendo come un *ensemble* implicito di reti neurali addestrate in parallelo, prevenendo massicciamente l'overfitting.

4.4 Autoencoders

Domande d'esame raccolte:

- Briefly describe what is the architecture of an autoencoder and its purpose. Draw an example.
- Explain the difference between Principal Component Analysis and Autoencoders in dimensionality reduction.

Linea Guida di Risoluzione (Rif. Slide 15 - Dimensionality Reduction):

- **Architettura & Scopo:** Un Autoencoder è una rete neurale formata da due sotto-reti: un **Encoder** e un **Decoder**. L'Encoder comprime l'input ad altissima dimensionalità (es. un'immagine) in uno strato nascosto a bassissima dimensionalità chiamato "bottleneck" (o spazio latente). Il Decoder tenta di ricostruire l'input originale a partire dallo spazio latente. Viene addestrato in modo non-supervisionato (il target $\mathbf{t}$ è uguale all'input $\mathbf{x}$) tramite MSE (Reconstruction Loss). Scopo: **Riduzione di dimensionalità e Anomaly Detection.**

- **PCA vs Autoencoders:** La PCA applica una trasformazione *strettamente lineare* basata sugli autovettori della covarianza. Gli Autoencoders, grazie alle funzioni di attivazione (es. ReLU, Sigmoid), possono imparare proiezioni su manifold **altamente non-lineari**, rappresentando un "Non-linear PCA".

5 Ensemble Methods & Instance-Based Learning

5.1 Ensemble: Bagging, Boosting e AdaBoost

Domande d'esame raccolte:

- Describe the concept of bagging and boosting in the definition of an ensemble model. Write down the formal equations to combine predictions.
- Discuss the difference between bagging and voting.
- Describe the general approach of AdaBoost and provide details on its sequential training.
- According to Bias-Variance Decomposition, explain if Bagging reduces Bias or Variance.

Linea Guida di Risoluzione (Rif. Slide 13 - Multiple Learners):

- **Bagging (Bootstrap Aggregation):** Crea M dataset estraendo punti dal dataset originale tramite **campionamento casuale con rimpiazzo** (Bootstrap). Addestra M modelli indipendenti e uguali (es. Alberi profondi) in **parallelo** sui dataset. Si uniscono le predizioni tramite maggioranza/media: $y_{\text{bagging}}(x) = \text{sign}(\frac{1}{M} \sum y_m(x))$. Lo scopo del Bagging è ridurre la **Varianza** (evita l'overfitting).
- **Voting puro:** A differenza del Bagging, nel Voting si addestrano **modelli strutturalmente diversi** (es. una SVM, una NN e un Decision Tree) sullo **stesso** dataset.
- **Boosting (AdaBoost):** Addestra una sequenza di *Weak Learners* (modelli molto semplici ad alto Bias, es. alberi di profondità 1). I modelli vengono addestrati in **sequenza**. Ad ogni passo, i campioni che sono stati misclassificati dal modello precedente ottengono un peso (*weight*) maggiore, forzando il classificatore successivo a concentrarsi su di essi.
- **AdaBoost Algoritmo:** Modello combinato finale calcolato come combinazione lineare pesata in base all'affidabilità (α_m) calcolata sull'errore di ogni weak learner: $Y_M(x) = \text{sign}(\sum_{m=1}^M \alpha_m y_m(x))$. L'obiettivo del Boosting è abbattere il **Bias** del modello (risolve underfitting).

5.2 K-Nearest Neighbors (K-NN)

Domande d'esame raccolte:

- Describe the K-nearest neighbors (K-NN) algorithm for classification.
- Given a 2D dataset for two classes, determine the answer of K-NN for a query point for $K=1$, $K=3$, and $K=5$ graphically.
- Is it more likely to have overfitting when K is low or when it is high? Motivate.

Linea Guida di Risoluzione (Rif. Slide 10 - Instance-based learning):

- **Algoritmo:** È un modello *non parametrico* che archivia in memoria l'intero dataset (*lazy learning*). Data una query $\mathbf{x}$, si calcola la distanza (solitamente Euclidea) rispetto a tutti i punti di addestramento D . Si selezionano i K campioni con la distanza minore. Si assegna a $\mathbf{x}$ la **classe maggioritaria** (più frequente) presente tra i suoi K vicini.

- **Grafico:** Metti il dito sul punto di query indicato e traccia mentalmente un cerchio crescente. Fermati quando il cerchio ingloba K punti del dataset. Conta di che classe sono e predici in base alla maggioranza.
- **Overfitting vs K:** L'overfitting (alta varianza) avviene quando K è molto **basso** (es. $K = 1$). A $K = 1$ il modello si adatta perfettamente alle anomalie e al rumore (creando confini di decisione super complessi noti come poligoni di Voronoi). Aumentare K crea confini più morbidi e tolleranti agli outlier.

6 Performance Evaluation

6.1 K-Fold Cross Validation

Domande d'esame raccolte:

- Describe with pseudo-code the K-Fold Cross Validation method to estimate the accuracy of a learning algorithm L on a dataset D .
- Describe how the method can be extended to compare two different learning algorithms L_A and L_B .

Linea Guida di Risoluzione (Rif. Slide 2 - Performance Evaluation):

- **Algoritmo Pseudo-Codice:**

```

K-Fold(Algorithm L, Dataset D, integer K):
    Divide D randomly into K equal disjoint subsets S_1, S_2, ... S_k
    Sum_errors = 0
    For i = 1 to K:
        Test_Set = S_i
        Train_Set = D \ S_i
        Model h = Train algorithm L on Train_Set
        Error = evaluate h on Test_Set
        Sum_errors = Sum_errors + Error
    Return (1 - (Sum_errors / K)) // Returns estimated Accuracy

```

- **Comparare due Algoritmi:** Al fine di confrontare in modo statisticamente robusto due algoritmi L_A e L_B , a ogni iterazione del loop calcoliamo l'errore commesso da L_A e da L_B sullo *stesso identico* Test_Set S_i . Definiamo $\delta_i = Error(L_A) - Error(L_B)$. Alla fine si valuta la media di tutte le δ_i . Se $\bar{\delta} < 0$, allora in media L_A produce un errore minore di L_B , deducendo che L_A è preferibile.

6.2 Confusion Matrix, Accuracy, Precision, Recall

Domande d'esame raccolte:

- Provide the definition of Confusion matrix for a multi-class classification problem. Explain the content of cells $C_{i,i}$ and $C_{i,j}$.
- Provide a numerical example of a non-symmetric confusion matrix for an unbalanced dataset. Compute accuracy, precision and recall.
- Discuss the statement: "Accuracy is not always a good performance metric".

Linea Guida di Risoluzione (Rif. Slide 2):

- **Confusion Matrix:** Matrice quadrata usata per visualizzare i risultati predittivi. La riga i indica la classe Reale (True Class), la colonna j indica la classe Predetta dal modello.

- *Cella $C_{i,i}$ (sulla Diagonale Principale):* Contiene il numero di istanze correttamente classificate (Istanze che appartengono realmente a i e predette come i).
- *Cella $C_{i,j}$ (Fuori dalla Diagonale):* Evidenzia gli errori. Contiene il numero di istanze che appartengono realmente alla classe i ma che il modello ha scambiato (misclassificato) come appartenenti alla classe j .
- **Accuracy limitations:** Se un dataset ha 99 campioni di classe "Normale" e 1 campione di "Cancro", un classificatore difettoso che risponde *sempre* "Normale" raggiungerà il 99% di Accuratezza, ma fallirà totalmente nell'identificare il Cancro (0% Recall). Pertanto, l'accuratezza devia le stime su dataset enormemente **sbilanciati**.
- **Formule:**

$$\text{Accuracy} = \frac{\text{Traccia}(C)}{\text{SommaTuttiElementi}} \quad \text{Precision}(\text{Class 2}) = \frac{TP}{TP + FP} \quad \text{Recall}(\text{Class 2}) = \frac{TP}{TP + FN}$$

7 Unsupervised Learning: Clustering & Dimensionality Reduction

7.1 K-Means Algorithm

Domande d'esame raccolte:

- Describe the K-means algorithm with pseudo-code, including inputs, outputs, main steps, and termination condition.
- Simulate the execution of K-means on a 2D data set qualitatively.
- Draw a dataset and a K-means solution considered "not a good" solution. Explain why it fails.

Linea Guida di Risoluzione (Rif. Slide 14 - Unsupervised Learning):

- **Pseudo-codice K-Means:**

```

Input: Dataset D = {x_n} of unlabeled points, integer K.
Output: K centroids (means) \mu_1, ... \mu_K.
1. Initialize \mu_1, ... \mu_K choosing K random points in D.
2. While (Termination Condition not met):
    // E-Step: Assignment
    For each x_i in D:
        Assign x_i to the cluster of the closest centroid \mu_j
    // M-Step: Update
    For each cluster j=1 to K:
        Recompute \mu_j as the Center of Mass (mean) of all points assigned to it.
3. Termination Condition: Algorithm stops when no centroid
   has moved during the last M-step (assignments no longer change).
```

- **Disegno della Simulazione:** Se l'esercizio ti dà due centroidi ($K = 2$) indicati con un cerchio, per il primo step traccia mentalmente un asse a metà della distanza tra i due centroidi e colora i punti a sinistra del primo colore e a destra del secondo. Poi sposta il centroide "dentro" il blocco denso dei punti che gli sono stati appena assegnati. Ripeti finché non si ferma.
- **Failures ("Not a good solution"):** K-Means fa assunzioni forti sulle geometrie: assume cluster di forma sferica ed equivalenti in densità. Fallisce pesantemente su set non convessi. *Esempio da disegnare:* Due forme a semi-luna (*Swiss Roll* o anelli concentrici). K-means le spaccherà brutalmente a metà con una riga dritta, rovinando l'identificazione strutturale dei due anelli.

7.2 Gaussian Mixture Models (GMM) e EM

Domande d'esame raccolte:

- Define the Gaussian Mixture Model (GMM) and describe its parameters.
- Describe the difference between K-Means and Expectation-Maximization (EM).

Linea Guida di Risoluzione (Rif. Slide 14):

- **GMM Definition:** Assume che i dati del dataset, pur non avendo etichette, siano stati originati (*generative model*) da K distribuzioni Normali multivariate mescolate. $P(\mathbf{x}) = \sum_{k=1}^K p_k \mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k)$. I parametri sono le probabilità a priori del cluster (p_k), i vettori delle medie (μ_k), e le matrici di covarianza (Σ_k).
- **K-Means vs EM:** K-means fa *hard clustering*, un punto o appartiene al cluster A o al cluster B. Usa solo la metrica della distanza (e quindi della Media) assumendo la stessa varianza per tutti. Il GMM risolto con *Expectation-Maximization* (EM) fa *soft clustering*: ad ogni punto viene assegnata la **probabilità** di appartenere al cluster A e la prob. di appartenere a B. Inoltre l'EM modella anche le **matrici di Covarianza** per potersi adattare a cluster di forma ellittica, stirata o molto densa.

7.3 Principal Component Analysis (PCA)

Domande d'esame raccolte:

- Describe how Principal Components are identified based on the principle of variance maximization.
- Describe how to compute PCA efficiently for high-dimensional data (e.g., $N \ll d$).
- Given binary images of 32×32 , each obtained applying 5 linear transformations on a base image. What is the data space dimensionality and intrinsic dimensionality? Is M (components) equal to intrinsic?

Linea Guida di Risoluzione (Rif. Slide 15 - Dimensionality Reduction):

- **Max Variance Principle:** Si cerca un vettore unitario $\mathbf{u}_1$ su cui proiettare i punti (centrati) $\mathbf{x}_n$, tale che la varianza del dataset proiettato sia massima. Matematicamente si deve risolvere:

$$\max_{\mathbf{u}_1} \mathbf{u}_1^T S \mathbf{u}_1 \quad \text{s.t.} \quad \mathbf{u}_1^T \mathbf{u}_1 = 1$$

dove S è la matrice di covarianza. Usando i moltiplicatori di Lagrange si trova la soluzione $S\mathbf{u}_1 = \lambda_1 \mathbf{u}_1$. La componente principale $\mathbf{u}_1$ non è altro che l'**autovettore della matrice di covarianza S associato all'autovalore λ_1 più grande**.

- **PCA Efficiente ($N < d$):** Su immagini 32×32 la dimensione è $d = 1024$. La matrice S ($X^T X$) sarebbe enorme (1024×1024). Essendo i campioni N molto pochi (es. $N = 10$), si calcolano prima gli autovalori sulla matrice invertita data-centered XX^T (grande solo 10×10 , computazionalmente triviale). Gli $N - 1$ autovalori trovati corrispondono a quelli reali di $X^T X$, perciò la conversione è rapidissima.
- **Dimensioni Intrinseche vs Data space:** Lo spazio ha $32 \times 32 = 1024$ dimensioni totali. Tuttavia, se le immagini sono ottenute da *una sola* immagine traslata/ruotata con 5 gradi di libertà, i dati "vivono" all'interno di una varietà ristretta a soli 5 gradi liberi (*intrinsic dimensionality*). Sfortunatamente, essendo la PCA un metodo strettamente **lineare**, fallirà nell'incollare la manifold curvata, e quindi M (componenti usati dalla PCA) **sarà maggiore** della dimensione intrinseca.

8 Reinforcement Learning (RL) & MDPs

8.1 Markov Decision Process (MDP) Model

Domande d'esame raccolte:

- Describe a complete model for a problem based on MDP, specifying all its elements.
- Describe the Markov property of Markovian models representing dynamic systems.
- Describe the difference between a Markov Decision Process (MDP) and a Partially Observable Markov Decision Process (POMDP).

Linea Guida di Risoluzione (Rif. Slide 17 & 19 - MDPs):

- **Definizione e Elementi:** Un MDP è definito matematicamente tramite la tupla $\langle X, A, \delta, r \rangle$:
 - X : L'insieme degli Stati dell'ambiente.
 - A : L'insieme delle Azioni che l'agente può compiere.
 - $\delta(x, a) \rightarrow X$: La funzione di Transizione. Può essere deterministica ($x' = \delta(x, a)$) o probabilistica/stocastica ($P(x'|x, a)$).
 - $r(x, a) \rightarrow \mathbb{R}$: La funzione di Reward che l'agente riceve.
- **Proprietà di Markov:** Asserisce che l'evoluzione futura del sistema dinamico dipende **unicamente** dallo stato corrente x_t e dall'azione corrente a_t , e **non** dalla storia passata che ha portato l'agente a quello stato. Tutto lo storico necessario a predire il futuro è incapsulato in x_t .
- **MDP vs POMDP vs HMM:** Nell'HMM (Hidden Markov Model) gli stati ci sono ma non sono osservabili dall'agente, il quale possiede un modello passivo senza possibilità di fare "Azioni". Nel **MDP standard**, la funzione di transizione per passare da uno stato a un altro può essere rumorosa/probabilistica, ma **una volta atterrato nello stato, l'agente ha Visibilità Piena di dove si trova**. Nel **POMDP** non solo la transizione è stocastica, ma la sensistica è difettosa: lo stato effettivo in cui si atterra è **parzialmente inosservabile** e l'agente rileva solo l'ambiente circostante tramite un modello probabilistico di osservazione $O(x', a, z) = P(z|x', a)$.

8.2 Q-Learning e Apprendimento

Domande d'esame raccolte:

- Formally describe the Reinforcement Learning procedure to compute the optimal policy (Q-Learning).
- Describe the main steps of a RL algorithm and provide the training rule to use to learn the best behavior (stochastic or deterministic).
- Describe the difference between SARSA and Q-Learning (On-policy vs Off-policy).

Linea Guida di Risoluzione (Rif. Slide 17):

- **Obiettivo in RL:** A differenza dell'algoritmo di Planning su MDP (dove δ ed r sono matematicamente noti a priori), nel Reinforcement Learning l'agente *esplora* fisicamente l'ambiente non sapendo a cosa porterà un'azione. Deve determinare la policy ottima $\pi^*(x) = \operatorname{argmax}_a Q(x, a)$ che **massimizzi la cumulative discounted reward** nel tempo.
- **Q-Learning Training Rule (Deterministica):** Quando prova un'azione a dallo stato x , raccoglie l'osservazione futura x' e il reward r . Aggiorna il suo Q-Table in memoria tramite:

$$\hat{Q}(x, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(x', a')$$

- **Q-Learning Training Rule (Non Deterministica / Stocastica):** Poiché un'azione può fallire nel tempo (pavimento scivoloso), l'agente stila una stima media mobile aggiungendo un parametro α (learning rate):

$$\hat{Q}(x, a) \leftarrow (1 - \alpha)\hat{Q}(x, a) + \alpha \left[r + \gamma \max_{a'} \hat{Q}(x', a') \right]$$

- **Q-Learning vs SARSA:** Il *Q-Learning* usa la formula $\max_{a'} Q(x', a')$, basando l'aggiornamento sull'assunzione che l'azione futura sarà sempre quella teoricamente migliore. Per questo viene chiamato approccio **Off-Policy**. *SARSA* invece attende che la funzione di controllo scelga materialmente la mossa futura $\tilde{a}'$ (che potrebbe essere errata a causa dell'esplorazione casuale), e aggiorna il nodo con l'equazione $r + \gamma \hat{Q}(x', \tilde{a}')$. Per questo è un **On-Policy** learning.

8.3 Exploitation vs Exploration e k-Armed Bandit

Domande d'esame raccolte:

- Discuss the strategy for balancing exploration and exploitation in Reinforcement learning.
- Describe the k-armed bandit problem (also known as One-state MDP) with stochastic Gaussian behavior and unknown reward functions. Define problem and solution concept.

Linea Guida di Risoluzione (Rif. Slide 17):

- **Exploitation vs Exploration:** L'**Exploitation** (Sfruttamento) porta l'agente a usare la policy correntemente stimata per trarre il massimo profitto basato sulle stime conosciute: l'agente prende la strada di cui conosce già le garanzie. L'**Exploration** (Esplorazione) impone all'agente di selezionare mosse subottimali e ignote (es. in modo casuale) per mappare altre zone della mappa e cercare tracciati molto più favorevoli mai incontrati. Evita che l'agente si blocchi in minimi locali (soluzioni funzionanti, ma scarse).
- **La strategia Epsilon-Greedy:** In fase di operazione, l'agente tirerà un dado ad ogni mossa: con probabilità $1 - \epsilon$ sceglie la mossa migliore del Q-table, con probabilità ϵ sceglierà una mossa a caso (esplorando). Nel tempo, ϵ viene decrementato matematicamente affinché il robot sfrutti progressivamente quanto imparato.
- **k-Armed Bandit (One-State MDP):** Situazione matematica equivalente a "Slot Machine con K leve". È un MDP con spazio X composto da **Un Singolo Stato**. $A = \{a_1 \dots a_k\}$. Transizione porta sempre all'unico stato. La funzione reward $r(a_i)$ è **ignota**, stocastica e distribuita con rumore Gaussiano ignoto $\mathcal{N}(\mu_i, \Sigma_i)$.
- **Soluzione:** Campionamento progressivo delle K azioni per stimare le medie $\hat{\mu}_i$ (utilizzando media iterativa). La policy ottima che l'algoritmo deve calcolare al termine dell'esplorazione è selezionare stabilmente e unicamente la leva j corrispondente a: $j = \operatorname{argmax}_i \hat{\mu}_i$.